🧠 ¿QUÉ ES EL SISTEMA DE INTELIGENCIA GEOTÉCNICA?

EL PROBLEMA QUE RESUELVE

Imagina que estás construyendo un puente en Piura. De repente llegan lluvias intensas (como El Niño Costero 2017). El suelo se debilita, se satura de agua, pierde resistencia. Nadie lo vio venir. La obra se paraliza. Los costos se disparan. Se pierden semanas.

Ese es exactamente el problema. Las empresas constructoras en Piura no tienen forma de anticipar cómo el clima va a afectar el terreno donde construyen. Reaccionan cuando el daño ya ocurrió, no antes.

El sistema resuelve esto: anticipa el riesgo, lanza alertas, y le dice al ingeniero qué hacer, antes de que pase algo malo.

🎯 QUÉ DEBE LOGRAR EL SISTEMA

Tres cosas concretas, que son exactamente las tres dimensiones que mide tu tesis:

1. Mejorar la eficiencia en la planificación Que los proyectos cumplan su cronograma y presupuesto. Hoy en Piura el 42% de obras tienen ampliaciones no previstas. El sistema debe reducir eso.

2. Mejorar la continuidad operativa Reducir las horas que una obra se paraliza por factores del terreno o clima. Si hoy un proyecto se detiene en promedio 48 horas por evento geotécnico, el sistema debe bajar ese número.

3. Mejorar la capacidad de respuesta Que cuando ocurra un evento inesperado, el ingeniero reaccione en minutos, no en días. El sistema genera la alerta y le dice exactamente qué hacer.

🔄 CÓMO FUNCIONA — EL FLUJO COMPLETO

Aquí está el corazón del sistema. Son 5 etapas que ocurren en cadena:

```
┌──────────────────────────────────────────────────────────────┐
│                                                              │
│ DATOS ENTRAN → SE PROCESAN → SE ANALIZA → SE ALERTA → SE ACTÚA │
│                                                              │
└──────────────────────────────────────────────────────────────┘
```

ETAPA 1 — RECOLECCIÓN DE DATOS 🌧️

El sistema consume datos de dos fuentes externas gratuitas, de forma automática cada cierto tiempo:

Datos climáticos (de APIs públicas):

- Temperatura actual y proyectada

- Precipitación (lluvia en mm)
- Humedad del ambiente
- Velocidad del viento

Datos geotécnicos (ingresados por el ingeniero o tomados de estudios previos):

- Tipo de suelo (arcilla, arena, limo, roca)
- Capacidad portante (cuánto peso aguanta el suelo)
- Índice de humedad del suelo
- Resistencia al corte
- Nivel freático (profundidad del agua subterránea)

Estos datos se combinan porque la clave del sistema es justamente eso: **el clima y el suelo juntos determinan el riesgo real**. La lluvia sola no te dice nada. El suelo solo tampoco. Juntos sí.

ETAPA 2 — PROCESAMIENTO E INTELIGENCIA 🤖

Aquí entra el cerebro del sistema. Con los datos combinados, se ejecuta un **modelo de machine learning** que hace dos cosas:

Predice el nivel de riesgo geotécnico:

BAJO → El terreno está estable. Continuar normal.

MEDIO → Hay señales de alerta. Aumentar monitoreo.

ALTO → Riesgo real de falla. Tomar medidas preventivas.

CRÍTICO → Paralizar inmediatamente. Riesgo de colapso.

Genera una recomendación específica: No solo dice "riesgo alto". Le dice al ingeniero exactamente qué hacer:

- "Reducir carga sobre el sector norte de la cimentación"
- "Instalar drenaje de emergencia en talud sur"
- "Suspender excavaciones hasta que baje la saturación del suelo"

El modelo aprende de datos históricos de Piura y mejora con el tiempo.

ETAPA 3 — GESTIÓN DE PROYECTOS 📋

En paralelo al análisis de riesgo, el sistema lleva el control operativo de cada proyecto:

- Qué actividades estaban planificadas vs cuáles se ejecutaron
- Cuántas horas se paralizó la obra y por qué causa

- Cuánto presupuesto se ha consumido vs lo planificado
- Cuál es el estado actual de cada zona de trabajo

Esta es la información que permite **medir el impacto del sistema** para tu tesis. Antes de usarlo y después de usarlo, los números hablan solos.

ETAPA 4 — ALERTAS EN TIEMPO REAL ⚡

Cuando el modelo detecta riesgo medio, alto o crítico, el sistema activa el módulo de alertas. Esto ocurre sin que nadie lo tenga que pedir, automáticamente.

La alerta llega al dashboard en tiempo real usando **WebSocket** — es decir, sin necesidad de recargar la página, la alerta aparece instantáneamente en la pantalla del ingeniero de supervisión.

Cada alerta contiene:

- Nivel de severidad (color: verde / amarillo / naranja / rojo)
- Zona del proyecto afectada
- Causa probable (ej: "Saturación por lluvia acumulada de 85mm en 24h")
- Recomendación de acción concreta
- Tiempo estimado de riesgo

ETAPA 5 — VISUALIZACIÓN Y REPORTES 📊

Todo lo anterior se muestra en un **dashboard web** que puede ver el ingeniero, el supervisor o el gerente del proyecto.

El dashboard tiene tres pantallas principales:

Pantalla 1 — Mapa en tiempo real: Un mapa de Piura donde se ven todos los proyectos activos. Los marcadores cambian de color según el nivel de riesgo. Si hay una zona crítica, aparece un círculo rojo pulsante en el mapa.

Pantalla 2 — Panel de control del proyecto: Métricas en tiempo real del proyecto: avance del cronograma, horas de paralización acumuladas, presupuesto consumido, historial de alertas recibidas y tiempo de respuesta a cada una.

Pantalla 3 — Reportes para la tesis: Comparativa pre/post implementación del sistema. Genera automáticamente las tablas que necesitas para demostrar el impacto: cuántas horas se paralizaba antes vs ahora, cuánto tardaba la respuesta antes vs ahora, qué tan bien se cumplía el cronograma antes vs ahora.

👥 QUIÉNES USAN EL SISTEMA

Hay tres tipos de usuarios con roles distintos:

Administrador (Django): Configura el sistema, registra nuevos proyectos, carga los estudios geotécnicos de suelo, gestiona usuarios. Accede al panel de administración interno.

Ingeniero de Supervisión: Ve el dashboard principal, recibe alertas en tiempo real, registra eventos ocurridos, anota las medidas correctivas tomadas y el tiempo que tardó en responder. Esta es la persona que genera los datos de medición de la tesis.

Gerente de Proyecto: Ve los reportes de alto nivel: cumplimiento de cronograma, sobrecostos evitados, comparativa antes/después.

📐 ARQUITECTURA SIMPLIFICADA (solo JS)

```
┌──────────────────────────────────┐
│      REACT VITE (Frontend)       │
│                                  │
│ Dashboard · Mapa · Alertas · KPIs │
└──────────────┬───────────────────┘
               │ HTTP + WebSocket
               ▼
┌──────────────────────────────────┐
│      NODE.JS + EXPRESS           │
│                                  │
│ API REST · WebSocket · Cron Jobs │
│ ML con brain.js / ml-matrix      │
└──────────────┬───────────────────┘
               │
               ▼
┌──────────────────────────────────┐
│  PostgreSQL + Redis (opcional)   │
└──────────────────────────────────┘
```

Todo en JavaScript. El modelo de ML se implementa en Node.js usando librerías como **brain.js** (redes neuronales en JS puro) o **ml-classify-text** / regresión logística simple. Nada de Python, nada de Flask. Un solo lenguaje de punta a punta.

🗓️ ORDEN DE CONSTRUCCIÓN

Si lo vas a construir de cero, este es el orden lógico:

Semana 1-2: Base de datos + modelos de datos

Semana 3-4: API Node.js (CRUD de proyectos, suelos, métricas)

Semana 5-6: Integración APIs climáticas (OpenWeatherMap gratis)

Semana 7-8: Motor de predicción de riesgo (brain.js)

Semana 9-10: Sistema de alertas con WebSocket

Semana 11-12: Frontend React — Dashboard + Mapa + Alertas

Semana 13-14: Módulo de métricas pre/post + reportes

Semana 15-16: Pruebas con los 25 proyectos + análisis estadístico

---

✅ QUÉ DEMUESTRA EL SISTEMA EN LA TESIS

Al final, comparas dos momentos con los mismos 25 proyectos:

Indicador	ANTES del sistema	DESPUÉS del sistema
Cumplimiento cronograma	~65%	~85% (meta)
Horas promedio paralización	~72h/evento	~20h/evento (meta)
Tiempo de respuesta a alertas	~4 horas	~30 minutos (meta)
Sobrecosto promedio	~23%	~8% (meta)

Esos números son tu hipótesis. El sistema los genera automáticamente. Tu prueba t de Student o Wilcoxon los valida. Y tienes tu tesis sustentada con evidencia real.

¿Quieres que ahora empecemos a construir el sistema módulo por módulo? ¿Por cuál prefieres comenzar: la base de datos, el backend Node.js, o el frontend React?